COSC 4368 Maze Solver Project
Second Check-In Report

Group 15 - Marlon Melara, Ahnaf Murshid, Israel Mummay, Brittany Picazo, Osinachi Enemuoh

March 23, 2026

Introduction

This report describes the second check-in for the maze solver project. At this stage, the main focus was building a working maze environment from the provided image, solving the maze without hazards, visualizing the solution path, and then integrating hazards into the environment.

The system now supports a full grid-based maze environment with movement constraints, pathfinding, hazard interaction, and visualization of both the maze layout and the agent's behavior.

Maze Representation

The maze is represented internally as a 64×64 logical grid. Each logical cell corresponds to a valid position the agent can occupy, while walls are stored as blocked movement edges between neighboring cells instead of as filled wall cells.

Two arrays are used to store the wall structure:

- `maze_walls_right.npy` — stores whether movement from a cell to the cell on its right is blocked
- `maze_walls_down.npy` — stores whether movement from a cell to the cell below it is blocked

This representation preserves the original maze layout while allowing efficient movement checks and graph-based search.

Maze Solving (BFS)

A Breadth-First Search (BFS) algorithm was used to solve the maze from the start position to the goal position.

BFS was selected because the maze is an unweighted grid environment, so BFS guarantees a shortest path in terms of number of moves. It is also simple, deterministic, and well suited for validating whether the maze representation is correct.

The solver produced the following results:

- Path length: 423 cells

- Total moves required: 422
- Turns required (maximum of five actions per turn): 85

These results confirmed that the reconstructed maze was connected correctly and that the movement logic in the environment was functioning as expected.

Visualization

The maze and the BFS solution path were visualized using `matplotlib`. The extracted wall structure was drawn directly from the logical wall representation, and the BFS path was highlighted in red so the traversal from the start to the goal was easy to interpret.

Figure 1 shows the maze without hazards. Figure 2 shows the BFS solution path over the maze.

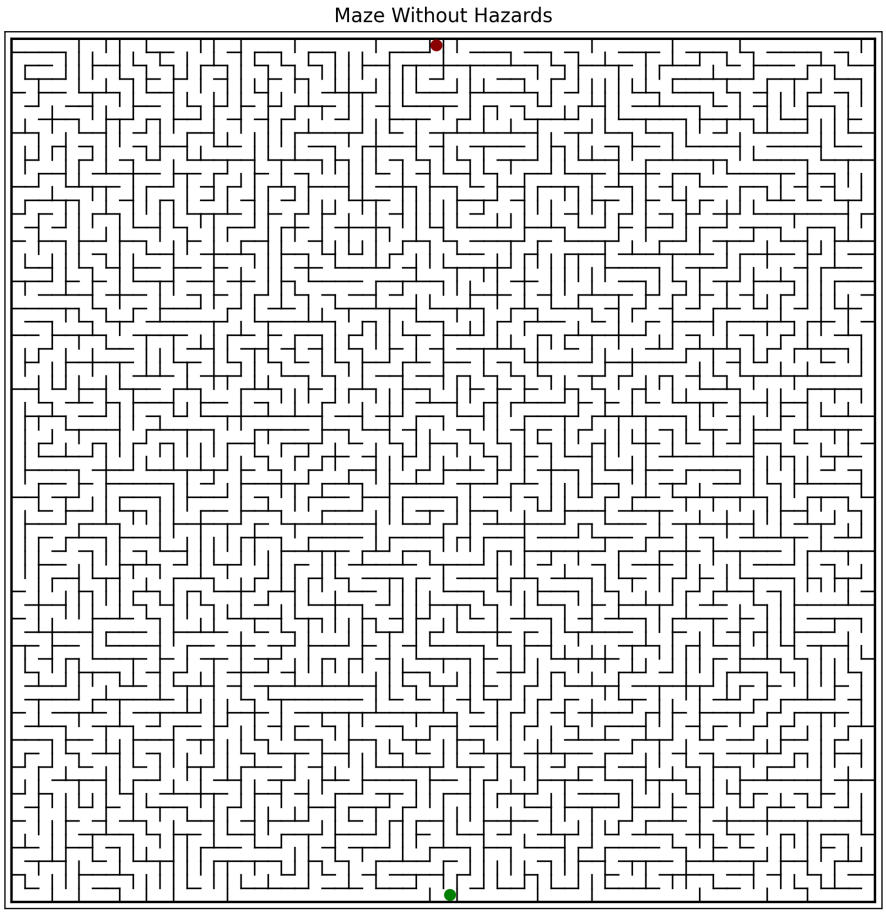

Figure 1: Maze loaded without hazards.

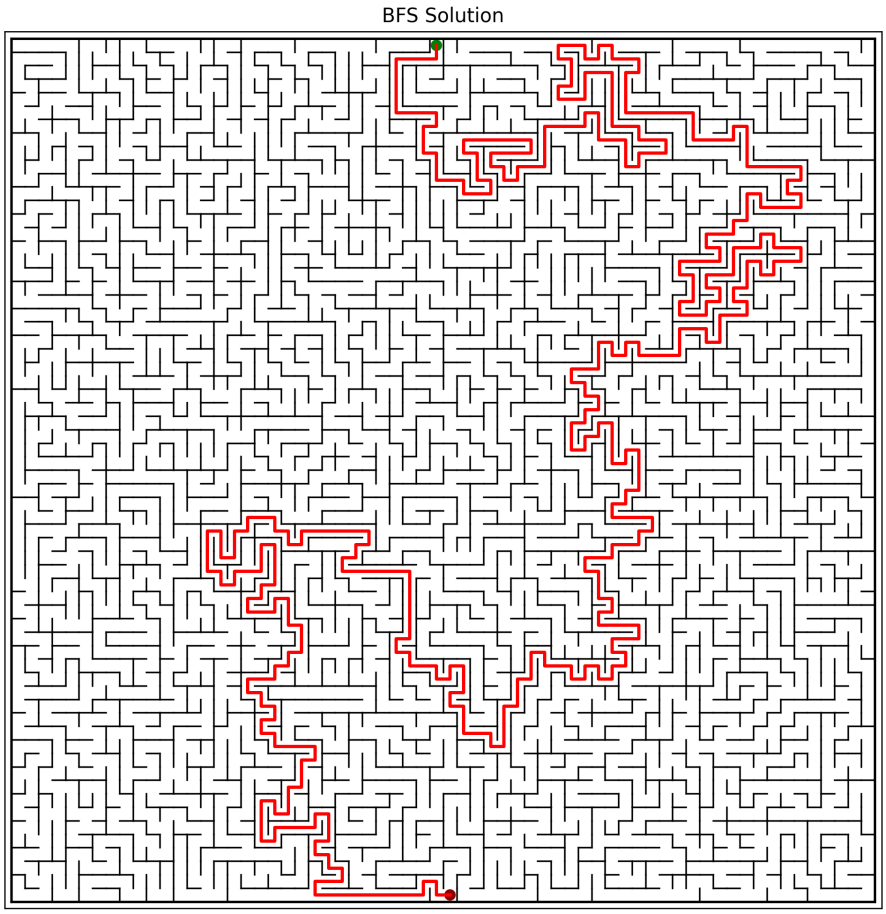

Figure 2: BFS solution path visualized on the maze.

Hazard System

After validating the maze and solver, hazards were added to the environment to introduce more complex behavior. Three hazard types were implemented:

- **Death pits** — the agent immediately dies upon entering the cell and respawns at the start location on the next turn
- **Teleport pads** — the agent is moved instantly to a predetermined destination cell
- **Confusion pads** — the agent's controls are inverted for the remainder of the current turn and the following turn

Hazards were generated automatically at approximately 7% of navigable cells, which satisfies the required density range of 5–10%. During hazard generation, a valid path between the start and goal positions was preserved so that the maze remained solvable.

Figure 3 shows the maze with hazards displayed.

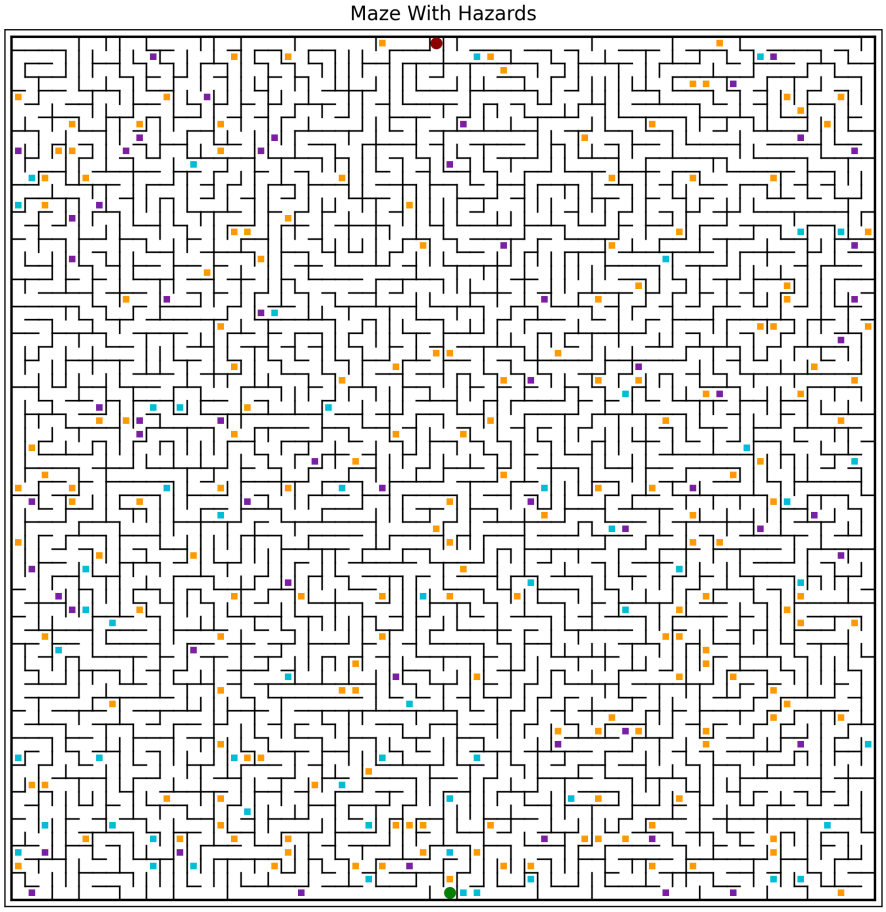

Figure 3: Maze with hazards loaded and displayed.

Environment Behavior

The hazard system was validated through scripted demonstrations showing each hazard type in action. These demonstrations included:

- stepping onto a death pit and respawning at the start
- stepping onto a teleport pad and relocating to its mapped destination
- stepping onto a confusion pad and observing temporarily inverted movement

The environment follows the required project turn format:

- each turn accepts between 1 and 5 actions
- actions are executed sequentially
- remaining actions are ignored if the agent dies or reaches the goal
- the environment returns a `TurnResult` object containing the resulting state information

This behavior allows the environment to function as a complete grid-world simulation rather than only as a static maze solver.

AI Usage Disclosure

AI assistance was used during development.

Tool used: ChatGPT

Prompts used:

- "Help me finish my second check in. I've already coded out the maze construction from the picture. I need to do the rest."
- "Give me the full block code for `maze.py` and let me know if I need to update `maze_config.py`."
- "Let's finish all the other files now with the corrected `maze.py` code and have a working demo check in."

How AI performed: The AI helped with scaffolding for the environment, solver, hazard generation, visualization, and report drafting. All generated code was reviewed, edited, and integrated by the team.

Conclusion

The second check-in successfully implemented the maze environment, BFS solver, visualization system, and hazard mechanics. The system now behaves as a complete grid-world environment that can demonstrate agent traversal and hazard interactions in a structured and visual way.

This provides a strong foundation for future work involving more advanced AI agents, hazard-aware navigation, and learning-based approaches.